28-APRIL-2026

Vulnerability Assessment Report

TARGET: http://demo.testfire.net

LAKSHMI PAVAN AKULA

TASK-01

ABOUT THE PROJECT

- This report presents a professional vulnerability assessment of a public demo website.
- The purpose of this assessment is to identify common security weaknesses without exploiting them and to provide clear, business-friendly recommendations.
- The analysis focuses on identifying risks that could impact the security, privacy, and reliability of the application.

OBJECTIVE

The objective of this assessment is to
• Identify security vulnerabilities in the website
• Classify risks (Low / Medium / High)
• Explain issues in simple language
• Provide practical remediation steps
• Perform analysis in a safe, read-only manner

SCOPE

- Only publicly accessible pages were tested
- No login bypass or attacks performed
- Passive scanning methods used
- No damage or disruption caused to the system

TOOLS USED

- Nmap → Port scanning and service detection
- OWASP ZAP → Passive vulnerability scanning
- Browser Developer Tools → Header inspection

NMAP RESULTS (PORT SCAN)

Finding: Open Ports Detected

Issue:

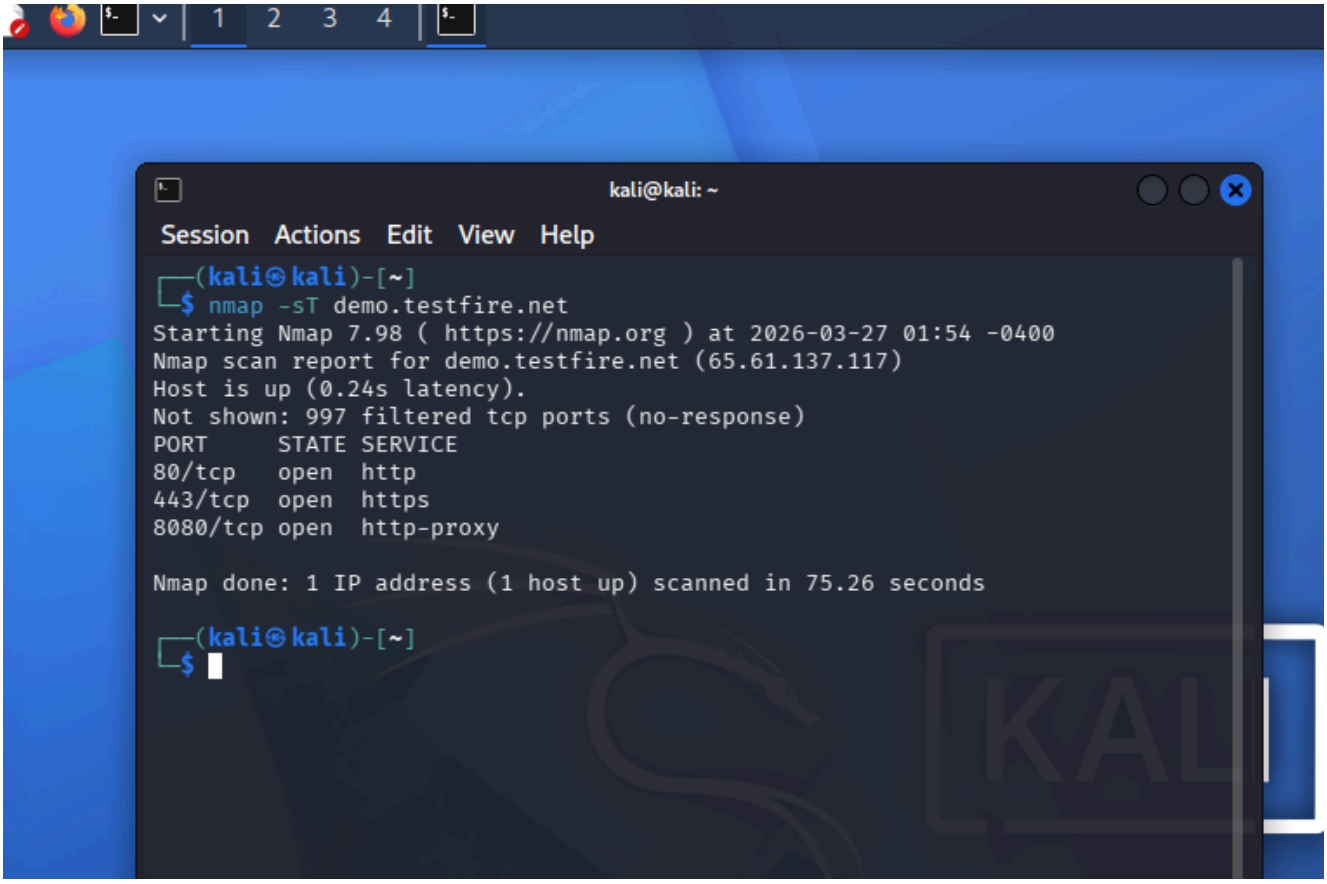
- Ports 80 (HTTP)
- 443 (HTTPS)
- 8080 (HTTP Proxy) are open

Why it matters:

- Open ports expose services that attackers may target.
- **Risk Level**: Medium

Remediation:

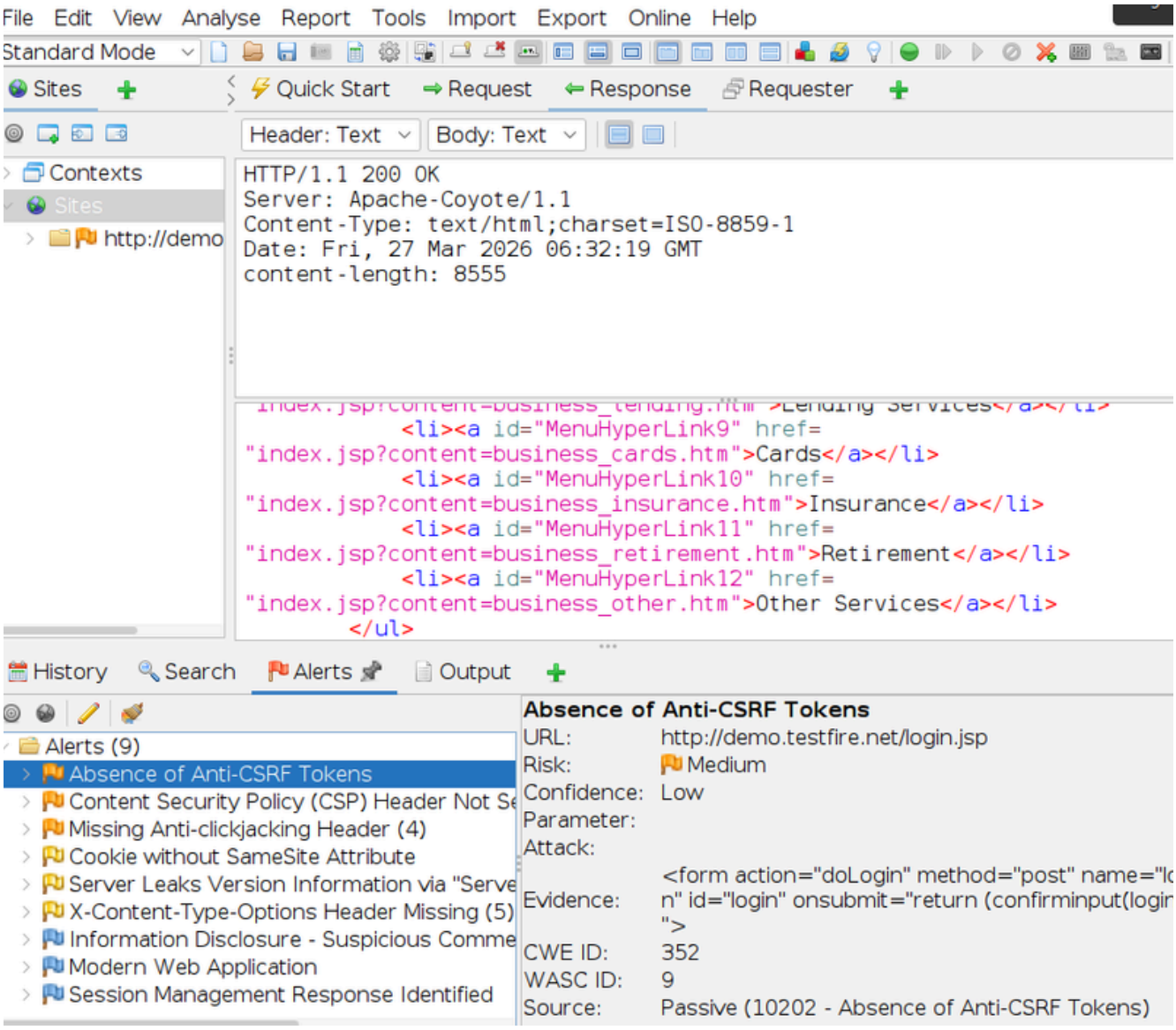
- Close unused ports
- Restrict access using firewall rules
- Monitor exposed services regularly

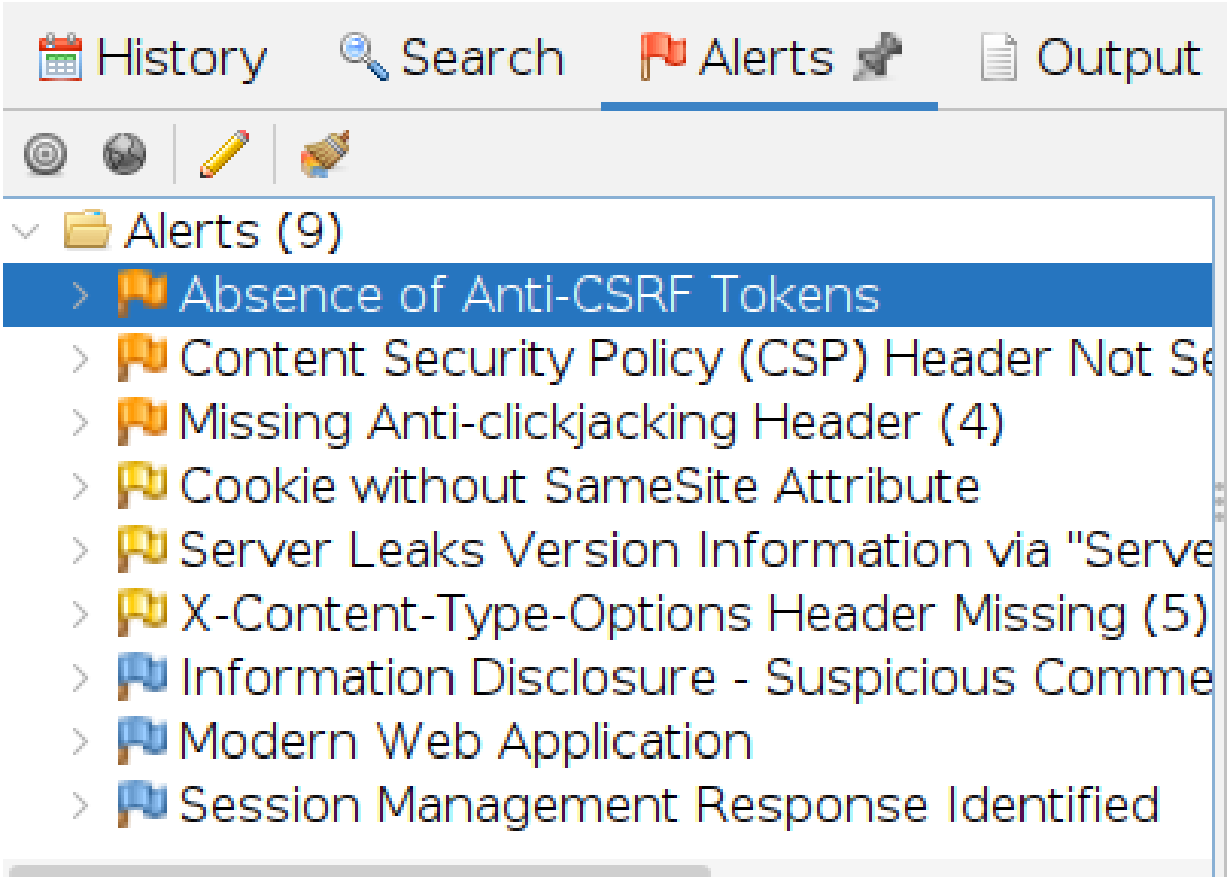


ZAP SUMMARY

Findings:

- Absence of Anti-CSRF Tokens
- Content Security Policy Header Not Set
- Missing Anti-clickjacking Header
- Cookie without SameSite Attribute
- Server Leaks Version Information
- X-Content-Type-Options Header Missing
- Information Disclosure
- Session Management Response Identified





1. Absence of Anti-CSRF Tokens

Meaning:

The website does not protect forms against unauthorized requests.

Why it matters:

Attackers can trick users into submitting requests (like transferring money or changing data) without their knowledge.

Risk Level: Medium

Remediation:

- Implement CSRF tokens in all forms
- Validate requests on the server-side

Use secure frameworks that include CSRF protection

2. Content Security Policy (CSP) Not Set

Meaning:

The website does not restrict which scripts can run in the browser.

Why it matters:

Attackers can inject malicious scripts, leading to Cross-Site Scripting (XSS) attacks.

Risk Level: Medium

Remediation:

- Add a Content-Security-Policy header
- Allow only trusted domains for scripts
- Block inline and unknown scripts

3. Missing Anti-clickjacking Header

Meaning:

The website can be embedded inside another site using iframes.

Why it matters:

Users can be tricked into clicking hidden buttons (clickjacking attack).

Risk Level: Medium

Remediation:

- Add X-Frame-Options: DENY or SAMEORIGIN
- Use Content-Security-Policy: frame-ancestors
- Prevent embedding in external websites

4. Cookie without SameSite Attribute

Meaning:

Cookies are sent even in cross-site requests.

Why it matters:

This increases the risk of session hijacking and CSRF attacks.

Risk Level: Low

Remediation:

- Set SameSite=Strict or Lax
- Enable Secure and HttpOnly flags
- Use HTTPS for all cookies

5. Server Version Disclosure

Meaning:

The server reveals its version (e.g., Apache-Coyote/1.1).

Why it matters:

Attackers can identify known vulnerabilities for that version.

Risk Level: Low

Remediation:

- Hide or remove server version from headers
- Configure server settings (e.g., ServerTokens Prod)

Regularly update server software

6. X-Content-Type-Options Missing

Meaning:

The browser may guess file types instead of strictly following headers.

Why it matters:

This can allow execution of malicious files (MIME sniffing attack).

Risk Level: Medium

Remediation:

- Add header: X-Content-Type-Options: nosniff
- Ensure correct content-type is configured
- Validate file uploads

7. Information Disclosure

Meaning:

Sensitive information (like comments or internal data) is exposed.

Why it matters:

Attackers can use this information to understand the system and find weaknesses.

Risk Level: Low

Remediation:

- Remove unnecessary comments from code
- Avoid exposing internal details
- Perform regular code reviews

8. Session Management Identified

Meaning:

The application uses session tracking mechanisms.

Why it matters:

Weak session management can lead to session hijacking.

Risk Level: Informational

Remediation:

- Use secure session cookies (Secure, HttpOnly)

Implement session expiration

RECOMMENDATIONS

- Implement security headers
- Add CSRF protection
- Secure cookies properly
- Hide server information
- Regularly monitor vulnerabilities

CONCLUSION

The application contains several security misconfigurations, including missing headers and lack of CSRF protection.

By applying recommended fixes, the overall security of the system can be significantly improved.